ShadowStack

Predictive Cloud Cost Optimizer

Product Backlog

Course	BCA 6th Semester – 3 Credits Project
Version	1.1
Project Start	February 7, 2026
Sprint 1 Deadline	**February 24, 2026**
Sprint 2 Deadline	**March 8, 2026**
Sprint 3 Deadline	**April 5, 2026**
Final Deadline	**April 15, 2026**
Methodology	Agile Scrum – 4 Sprints

Name	Student ID	Role
Aniruddha Bhide	CU23BSC001A	Project Lead & Backend Architect
Sanay Krishna	CU23BCA0049A	Frontend Developer & Visualization Engineer
Arpita M	CU23BCA0011A	Documentation & Quality Support Engineer

Sprint 1 deliverables sourced from the official ShadowStack Sprint 1 document, Feb 2026

Contents

1 Overview and Conventions **2**

2 Epic Summary **3**

3 Sprint 1 – Project Setup, Backend and Documentation **4**
3.1 Sprint Goal 4
3.2 Product Backlog Items 4

4 Sprint 2 – ML Pipeline, Data Engineering and API **7**
4.1 Sprint Goal 7
4.2 Product Backlog Items 7

5 Sprint 3 – Frontend Dashboard and Visualization **10**
5.1 Sprint Goal 10
5.2 Product Backlog Items 10

6 Sprint 4 – Integration, Testing and Final Delivery **13**
6.1 Sprint Goal 13
6.2 Product Backlog Items 13

7 Future Backlog (Post-MVP) **16**

8 Complete MVP Backlog Summary **17**

9 Success Metrics and Definition of Done **19**
9.1 Measurable Success Criteria 19
9.2 Definition of Done (DoD) 19
9.3 Live Demo — Required Scenarios 19

Overview and Conventions

ShadowStack predicts infrastructure costs *before* code deployment using static code analysis and machine learning, surfacing cost predictions directly in GitHub pull request comments.

Sprint Timeline

Sprint	Theme	Start	Deadline
1	Project Setup, Backend & Documentation	Feb 7, 2026	**Feb 24, 2026**
2	ML Pipeline, Data Engineering & API	Feb 25, 2026	**Mar 8, 2026**
3	Frontend Development & Visualization	Mar 9, 2026	**Apr 5, 2026**
4	Integration, Testing & Final Delivery	Apr 6, 2026	**Apr 15, 2026**

Field Definitions

Field	Description
ID	Unique backlog item identifier (PBI-001 . . .)
User Story / Task	Written as: *As a [role], I want [goal], so that [benefit]*
Acceptance Criteria	Measurable conditions that confirm the item is done
Priority	**HIGH** = MVP-critical **MED** = MVP, not blocking **FUTURE** = Post-MVP
SP	Story Points on Fibonacci scale (1, 2, 3, 5, 8, 13)
Owner	Primary responsible team member

Epic Summary

Epic	Name	Description	Sprint(s)
EP-1	Infrastructure & DevOps	Docker config, repo setup, architecture diagram	1
EP-2	Backend Core	FastAPI, PostgreSQL, webhook, code parser, API endpoints	1 – 3
EP-3	Documentation & Agile Artefacts	SRS, use-case diagrams, dataset schema, sprint tracking	1 – 4
EP-4	Frontend Foundation	React setup, wireframes, UI components, chart library	1
EP-5	ML Pipeline & Data Engineering	Synthetic data, feature engineering, model training	2
EP-6	Cost Estimation Engine	Code-to-cost mapping, pricing simulation	2
EP-7	RESTful API Layer	Prediction, history, and model metrics endpoints	2
EP-8	Frontend Dashboard & Charts	Full dashboard, charts, filters, real-time updates	3
EP-9	GitHub PR Integration	Automated PR comment with cost prediction	4
EP-10	Testing & QA	Unit, integration, system tests ($\geq$70% coverage)	1 – 4
EP-11	Final Delivery	End-to-end integration, performance, demo prep	4
EP-12	Future Enhancements	Deep learning, Kubernetes operator, multi-cloud	Post-MVP

Sprint 1 – Project Setup, Backend and Documentation

Duration: February 7 – February 24, 2026 (2.5 weeks)
Deadline: **February 24, 2026**
Goal: Project setup, backend architecture, and documentation

Sprint Goal

Three workstreams run in parallel. **Aniruddha** builds the backend foundation (FastAPI, PostgreSQL, Docker, GitHub webhook) and produces the architecture diagram and product backlog. **Sanay** initialises the React project, creates wireframes, builds basic UI components, and sets up the chart library. **Arpita** delivers the SRS, use-case diagrams, dataset schema, and all Agile tracking artefacts.

Product Backlog Items

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
Aniruddha Bhide — Backend & Infrastructure					
PBI-001	**[EP-1] System Architecture Diagram** As a team, I want a system architecture diagram in the repo so all components and data flows are defined before coding begins.	Shows: frontend, FastAPI, ML engine, DB, webhook, Docker layers; reviewed by team; committed as PNG/PDF	**HIGH**	2	Aniruddha
PBI-002	**[EP-2] FastAPI Setup with PostgreSQL** As a developer, I want FastAPI wired to PostgreSQL in Docker so subsequent features have a working foundation.	FastAPI on port 8000; `/health` returns 200; DB connected; migrations applied; `usage_data`, `predictions`, `model_metrics` tables exist	**HIGH**	5	Aniruddha
PBI-003	**[EP-2] GitHub Webhook Listener** As a system, I want `POST /webhook/github` to receive and validate PR events so code analysis is triggered automatically.	Accepts PR events; HMAC signature validated; payload logged to DB; responds within 30 s; tested with simulated payload	**HIGH**	5	Aniruddha
PBI-004	**[EP-1] Docker Configuration** As a developer, I want all services in Docker Compose so the stack runs on any laptop with one command.	`docker compose up` starts API, DB, frontend; containers healthy; networking verified; `.env.example` committed	**HIGH**	3	Aniruddha
PBI-005	**[EP-3] Product Backlog & Sprint Planning** As a team, I want a prioritised backlog on the Agile board so all members have clear assigned tasks.	All items in GitHub Projects/Trello; each has owner, SP, priority, sprint assignment; Sprint 1 board fully populated	**HIGH**	2	Aniruddha
Sanay Krishna — Frontend Foundation					

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
PBI-006	**[EP-4] React Project Initialization** As a frontend developer, I want a React 18 project with routing, linting, and env config so all UI work has a clean base.	Vite/CRA scaffold; React Router v6; Axios installed; `.env` for API URL; runs on port 3000 in Docker	HIGH	3	Sanay
PBI-007	**[EP-4] Dashboard Wireframes** As a designer, I want low-fidelity wireframes for all screens so the team shares a visual reference.	Covers: main dashboard, cost trends, service breakdown, model metrics views; approved by team; committed to repo	HIGH	2	Sanay
PBI-008	**[EP-4] Basic UI Components: Header, Sidebar, Navigation** As a user, I want a persistent header and sidebar with working routes so moving between views is intuitive.	Header with logo; sidebar links to all views; active route highlighted; mobile hamburger menu; renders without errors	HIGH	3	Sanay
PBI-009	**[EP-4] Chart Library Setup** As a developer, I want Chart.js/D3.js installed with a working sample chart so the visualisation layer is ready for data in Sprint 3.	Library installed; sample line chart renders with mock data; reusable wrapper component created; documented in README	MED	2	Sanay
Arpita M — Documentation & Agile Artefacts					
PBI-010	**[EP-3] Software Requirements Specification (SRS)** As an evaluator, I want a complete SRS so all system requirements are formally captured.	Covers: purpose, scope, functional & non-functional requirements, constraints, assumptions; formatted to academic standard; submitted as PDF	HIGH	3	Arpita
PBI-011	**[EP-3] Use-Case Diagrams** As a stakeholder, I want UML use-case diagrams covering all actors so scope is visually communicated.	Actors: Developer, Eng. Manager, Sys. Admin; use cases include: submit PR, view dashboard, monitor model; committed to repo	HIGH	2	Arpita
PBI-012	**[EP-3] Dataset Schema Design** As a data engineer, I want a documented DB schema so data collection and ML work are aligned from the start.	Covers `usage_data`, `predictions`, `model_metrics`; column names, types, descriptions listed; reviewed by Aniruddha	HIGH	2	Arpita
PBI-013	**[EP-3] Sprint Tracking Documents & Burndown Chart** As Scrum Master, I want sprint planning notes, daily standup logs, sprint backlog status, and a burndown chart maintained throughout Sprint 1.	Planning notes written; standup log updated daily; backlog shows To Do/In Progress/Done status; burndown chart plotted at sprint end	HIGH	2	Arpita

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
PBI-014	**[EP-10] Unit Tests: Backend Core Modules** As a developer, I want unit tests for FastAPI routes and DB connection so regressions are caught early.	Tests cover `/health`, `/webhook/github`, DB connection; ≥60% coverage on Sprint 1 modules; all pass in CI	MED	2	Aniruddha
PBI-015	**[EP-3] Sprint 1 Retrospective Notes** As a team, I want a retrospective doc recording what went well, what did not, and action items.	Held within 24 h of sprint end; covers: went well, improvement areas, action items; committed to repo	MED	1	Arpita
			Sprint 1 Total	**37**	

Table 1: Sprint 1 Product Backlog Items

Sprint 1 Success Criteria

- Backend running in Docker with FastAPI and PostgreSQL connected and healthy.
- GitHub webhook receives and validates PR events.
- React app initialised with navigation, UI components, and chart library working.
- SRS, use-case diagrams, and dataset schema completed and submitted.
- System architecture diagram committed to the repository.
- All Agile artefacts (backlog, standup logs, burndown chart) maintained.

Sprint 2 – ML Pipeline, Data Engineering and API

Duration: February 25 – March 8, 2026 (1.5 weeks)
Deadline: **March 8, 2026**
Lead: Aniruddha Bhide (ML & API), Arpita M (Data Support)

Sprint Goal

Build the complete ML pipeline from data generation through model training to a live prediction API. Deliver the Python AST code analyser, complexity scorer, cost simulation engine, and all API endpoints needed by the frontend. Arpita prepares synthetic datasets, preprocessing utilities, and test case documentation.

Product Backlog Items

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
Arpita M — Data Preparation & Documentation					
PBI-016	**[EP-5] Synthetic Dataset Generation (≥500 samples)** As a data engineer, I want generation scripts for labelled training samples so ML models have sufficient data.	≥500 records; columns: complexity score, resource patterns, monthly cost (USD); stored in PostgreSQL and CSV; reproducible with seed	HIGH	5	Arpita
PBI-017	**[EP-5] Historical Usage Data Collection & Formatting** As a data engineer, I want historical usage data ingested so time-series models train on realistic patterns.	Data loaded into `usage_data`; timestamps, service names, cost values present; data quality checks pass	HIGH	3	Arpita
PBI-018	**[EP-5] Data Preprocessing Utilities** As an ML engineer, I want normalisation and missing-value utilities so models receive clean inputs.	Normaliser, imputer, scaler implemented; integrated into feature pipeline; edge cases handled; unit tested	HIGH	3	Arpita
PBI-019	**[EP-3] Sprint 2 Test Case Documentation** As a QA engineer, I want test cases documented for Sprint 2 modules so validation is traceable.	Test cases for: data generation, feature extraction, each model, prediction API; expected vs. actual logged; stored in repo	MED	2	Arpita
Aniruddha Bhide — ML Pipeline, Code Analysis & API					
PBI-020	**[EP-2] AST-Based Python Code Parser** As a developer, I want Python files parsed via the AST module so code structure is extracted for analysis.	Accepts raw Python string; returns node counts, function list, import list; syntax errors handled; unit tests pass	HIGH	5	Aniruddha

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
PBI-021	**[EP-2] Cyclomatic Complexity Scoring Algorithm** As a developer, I want a complexity score (1–10) per file so complex code is flagged for cost review.	Score returned for any valid Python file; correlates with branch and loop count; formula documented; edge cases tested	HIGH	5	Aniruddha
PBI-022	**[EP-5] Feature Extraction Pipeline** As an ML engineer, I want a pipeline converting code metrics to normalised feature vectors so training and inference share the same preprocessing logic.	Accepts complexity metrics and usage data; outputs normalised vector; serialisable with `joblib`; unit tested	HIGH	5	Aniruddha
PBI-023	**[EP-5] Linear Regression Model Training** As an ML engineer, I want a trained Linear Regression baseline so a fast, explainable model is available.	Trained on 80/20 split; MAE and RMSE computed; results in `model_metrics` table; serialised to `.pkl`	HIGH	3	Aniruddha
PBI-024	**[EP-5] ARIMA Time-Series Model Training** As an ML engineer, I want a trained ARIMA model so time-dependent cost trends are captured.	Fitted on historical sequences; 30-day forecast horizon; MAE <15% on test set; model serialised	HIGH	5	Aniruddha
PBI-025	**[EP-5] Random Forest Ensemble Model Training** As an ML engineer, I want a trained Random Forest so an ensemble approach is available for comparison.	Trained with cross-validation; feature importances logged; MAE and RMSE computed; serialised	HIGH	5	Aniruddha
PBI-026	**[EP-5] Model Evaluation Framework (MAE, RMSE, R^2)** As a system admin, I want a unified evaluation framework so the best model is selected objectively.	Script runs on any serialised model; outputs MAE, RMSE, R^2; results in DB; best model flagged automatically	HIGH	3	Aniruddha
PBI-027	**[EP-5] Model Serialisation & Versioning** As a developer, I want models saved with version IDs so rollback and comparison are possible.	Saved as `model_v{n}_{type}.pkl`; version metadata in DB; API loads latest model at startup	MED	2	Aniruddha
PBI-028	**[EP-6] Cost Simulation Engine with Mocked Pricing** As a developer, I want a simulation engine with mocked pricing tiers so cost estimates are generated without live API access.	Config from YAML; compute, storage, network tiers defined; returns itemised breakdown per service; unit tested	HIGH	5	Aniruddha

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
PBI-029	**[EP-6] Basic Cost Estimation Logic** As a developer, I want complexity metrics mapped to monthly cost so every PR analysis produces an actionable figure.	Returns monthly cost (USD) given complexity score and patterns; mapping rules documented; range validated in tests	HIGH	3	Aniruddha
PBI-030	**[EP-7] Full RESTful API Endpoints** As a developer, I want all API routes implemented and returning real data so the frontend has stable contracts.	Live: `POST /api/predict`, `GET /api/costs/history`, `GET /api/model/metrics`; OpenAPI docs at `/docs`; P95 <500 ms	HIGH	5	Aniruddha
PBI-031	**[EP-10] Unit Tests: ML & Analysis Modules** As a developer, I want unit tests for the feature pipeline, cost estimator, and each model so regressions are caught in CI.	≥70% coverage on Sprint 2 modules; covers parser, scorer, extractor, estimator; all pass in CI	HIGH	3	Aniruddha
PBI-032	**[EP-3] Sprint 2 Retrospective & Standup Logs** As a team, I want daily logs and a retrospective at sprint end so Agile artefacts stay current.	Standup log updated daily; burndown chart updated; retrospective committed within 24 h of sprint end	MED	1	Arpita
			Sprint 2 Total	**63**	

Table 2: Sprint 2 Product Backlog Items

Sprint 2 Success Criteria

- Best model achieves MAE <15% on the held-out test dataset.
- `POST /api/predict` returns cost estimates within 2 seconds.
- At least 3 models trained, evaluated, and compared with documented results.
- Cost simulation engine returns itemised cost breakdown per service.
- AST code parser and complexity scorer fully functional and unit tested.

Sprint 3 – Frontend Dashboard and Visualization

Duration: March 9 – April 5, 2026 (3.5 weeks)
Deadline: **April 5, 2026**
Lead: Sanay Krishna

Sprint Goal

Complete the React dashboard by connecting it to live backend APIs and implementing all required interactive visualisations. Arpita documents frontend test cases and validation results.

Product Backlog Items

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
Sanay Krishna — Full Dashboard Implementation					
PBI-033	**[EP-8] API Integration Layer (Axios)** As a frontend developer, I want a centralised Axios service layer so all API calls have uniform error handling.	Axios instance with `.env` base URL; interceptors for 4xx/5xx; loading and error states per request; typed response models	HIGH	3	Sanay
PBI-034	**[EP-8] Cost Trend Line Chart** As a developer, I want an interactive line chart of historical and predicted costs so I can identify patterns.	Chart.js/D3 line chart; x-axis: date, y-axis: USD; predicted vs. actual series; tooltips; time-range selector (7d/30d/90d)	HIGH	5	Sanay
PBI-035	**[EP-8] Service-Wise Cost Breakdown Pie Chart** As a developer, I want a donut chart breaking costs by service so I can identify the most expensive components.	Donut with ≥3 segments; legend with percentages; tooltips; data from `/api/costs/history`; updates on filter change	HIGH	5	Sanay
PBI-036	**[EP-8] Predicted vs. Actual Cost Comparison Chart** As a developer, I want a grouped bar chart comparing predicted and actual costs so model accuracy is visually obvious.	Grouped bars per time bucket; variance percentage shown; colour-coded above/below threshold	HIGH	5	Sanay
PBI-037	**[EP-8] ML Model Performance Metrics Panel** As a system admin, I want a panel showing MAE, RMSE, and R^2 for all models so I can monitor prediction quality.	Metrics from `/api/model/metrics`; displayed as cards with colour thresholds; model version selector present	HIGH	3	Sanay
PBI-038	**[EP-8] Historical Trends with Time-Range Filters** As a user, I want date-range pickers on all charts so I can zoom into specific periods.	Date picker present; selecting range re-fetches all charts; default: last 30 days; filter state in URL params	MED	3	Sanay

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
PBI-039	**[EP-8] Interactive Data Filtering & Sorting** As a user, I want to filter cost data by service and sort records so I can focus on relevant data.	Service dropdown filter; sortable cost table; combined filter and sort works correctly	MED	3	Sanay
PBI-040	**[EP-8] Real-Time Dashboard Updates** As a user, I want the dashboard to refresh automatically so new PR analyses appear without a manual reload.	Polling every 30 s or WebSocket connection; new data reflected in charts; loading spinner shown	MED	3	Sanay
PBI-041	**[EP-8] Responsive UI & Accessibility** As a user, I want the dashboard usable on desktop and mobile so it is accessible anywhere.	Layout adapts 320 px to 1920 px; WCAG AA contrast; keyboard navigation; tested on Chrome and Firefox	MED	3	Sanay
PBI-042	**[EP-8] Team Cost Dashboard View** As an engineering manager, I want a team-wide cost overview so I can make informed budget decisions.	Dedicated view with aggregate predicted costs; filterable by date; cost totals card on main dashboard	MED	2	Sanay
Aniruddha Bhide — Backend Support					
PBI-043	**[EP-2] Cost History Retrieval Endpoint** As a frontend developer, I want `GET /api/costs/history` with pagination so charts display accurate historical data.	Filters by date range and service; pagination supported; response <500 ms for 1000 records	HIGH	3	Aniruddha
Arpita M — Testing Documentation					
PBI-044	**[EP-10] Frontend Unit Tests (Jest / RTL)** As a developer, I want Jest and React Testing Library tests for all key components so UI regressions are caught.	≥70% coverage on frontend components; chart render tests; API mock tests; all pass in CI	HIGH	3	Sanay
PBI-045	**[EP-3] Frontend Validation & Test Case Docs** As a QA engineer, I want test cases and results documented for Sprint 3 UI components.	Cases cover each chart, filter, nav link, and API call; expected vs. actual logged; stored in repo	MED	2	Arpita
PBI-046	**[EP-3] Sprint 3 Retrospective & Standup Logs** As a team, I want standup logs and a retrospective at sprint end so Agile artefacts stay current.	Daily logs updated; burndown chart updated; retrospective committed within 24 h of sprint end	MED	1	Arpita
			Sprint 3 Total	**47**	

Table 3: Sprint 3 Product Backlog Items

Sprint 3 Success Criteria

- All visualisations display real data from the live backend API.
- Page load time <2 seconds.
- Responsive design verified on desktop and mobile.
- All charts are interactive, labelled, and connected to live endpoints.
- Frontend unit test coverage ≥70%.

Sprint 4 – Integration, Testing and Final Delivery

Duration: April 6 – April 15, 2026 (1.5 weeks)
Deadline: **April 15, 2026 (Final Submission)**
Leads: Aniruddha (Integration), Arpita (QA/Docs), Sanay (UI Polish)

Sprint Goal

Deliver a fully integrated, tested, and documented system ready for academic evaluation. The end-to-end PR-to-dashboard workflow must be live. All documentation must be 100% complete and every measurable success criterion must pass.

Product Backlog Items

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
Aniruddha Bhide — Integration & Performance					
PBI-047	**[EP-9] GitHub PR Comment Automation** As a developer, I want the system to auto-post a cost-prediction comment on each PR so I get feedback without leaving GitHub.	On PR open/update, GitHub API posts markdown table (current cost, predicted, delta, recommendations); verified on test repo	HIGH	8	Aniruddha
PBI-048	**[EP-11] End-to-End Workflow: PR to Dashboard** As a developer, I want to open a PR and see results on the dashboard within 30 seconds so the full pipeline is operational.	PR triggered; webhook fires; analysis runs; comment posted; dashboard updates — all within 30 s; documented with screenshots	HIGH	8	Aniruddha
PBI-049	**[EP-11] Performance Optimisation & Profiling** As a developer, I want critical paths profiled and bottlenecks resolved so all time targets are met.	PR analysis <30 s; API <500 ms P95; dashboard load <2 s; profiling report committed to repo	HIGH	3	Aniruddha
PBI-050	**[EP-11] Final Docker Deployment Verification** As a developer, I want to confirm the full system starts from a clean clone with one command.	`git clone` + `docker compose up` gives a functional system on a fresh machine; verified by another team member	HIGH	2	Aniruddha
PBI-051	**[EP-3] API Documentation** As an evaluator, I want complete API docs with request/response examples for all endpoints.	All endpoints in OpenAPI; Postman collection exported; example requests and responses included	HIGH	2	Aniruddha
Arpita M — QA, Documentation & Final Report					

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
PBI-052	**[EP-10] Integration Tests: Critical Workflows** As a QA engineer, I want automated integration tests covering the webhook-to-prediction path.	Covers: webhook → analysis → DB write → API response; all pass in CI; results documented	HIGH	5	Arpita
PBI-053	**[EP-10] System Testing & Validation** As a QA engineer, I want the system validated against all measurable criteria so targets are demonstrably met.	Verified: MAE <15%, response <30 s, load <2 s, uptime ≥95%, coverage ≥70%; validation report submitted	HIGH	5	Arpita
PBI-054	**[EP-3] Deployment Guide & User Manual** As a user, I want step-by-step setup and usage documentation so evaluators can run ShadowStack independently.	Deployment guide: Docker setup, env vars, DB migration, test run; user manual: dashboard and GitHub integration	HIGH	2	Arpita
PBI-055	**[EP-3] Final Project Report Compilation** As a team, I want a consolidated final report combining all documentation and results for submission.	Includes: executive summary, architecture, implementation, ML results, test results, limitations, future scope	HIGH	3	Arpita
PBI-056	**[EP-10] Comprehensive Bug Fixing** As a team, I want all known issues resolved before submission so the system is stable during evaluation.	Zero open P1/P2 bugs; all issues closed; regression test added for each fix	HIGH	3	All
Sanay Krishna — UI Polish & Demo					
PBI-057	**[EP-11] UI/UX Refinements & Final Polish** As a user, I want the dashboard to feel polished and professional for evaluation.	Consistent colour scheme; no layout overflow; loading states on async ops; error states graceful; cross-browser verified	MED	2	Sanay
PBI-058	**[EP-11] Demo Preparation & Presentation Slides** As a team, I want a rehearsed demo and slide deck so we can present all 6 scenarios confidently.	Slides: problem, solution, architecture, demo, metrics, future scope; rehearsed ≥2 times; backup screenshots prepared	HIGH	3	All
PBI-059	**[EP-11] Code Cleanup & Refactoring** As a developer, I want dead code removed and naming standardised so the codebase is clean for submission.	No commented-out code; consistent naming (snake_case Python, camelCase JS); no untracked TODOs; linter passes	MED	2	All
PBI-060	**[EP-3] Sprint 4 Retrospective & Lessons Learned** As a team, I want a combined lessons-learned document from all four sprints in the final report appendix.	Sprint 4 retrospective held; all-sprints document committed; included in final report appendix	MED	1	Arpita

ID	User Story / Task	Acceptance Criteria	Priority	SP	Owner
			Sprint 4 Total	**49**	

Table 4: Sprint 4 Product Backlog Items

Sprint 4 Success Criteria

- Complete PR-to-dashboard workflow operational end-to-end within 30 seconds.
- All unit and integration tests passing with $\geq$70% coverage.
- System meets every measurable success metric.
- Documentation 100% complete and reviewed by all team members.
- Demo rehearsed and all 6 scenarios ready to demonstrate live.
- Code clean, committed, and ready for final academic submission.

Future Backlog (Post-MVP)

Items below are formally captured for future phases but are out of scope for the 10-week MVP.

ID	Feature	Description	Priority	SP	Phase
PBI-F01	LSTM Deep Learning Model	Sequential cost forecasting with LSTM networks	FUTURE	13	Phase 2
PBI-F02	XGBoost Gradient Boosting	Higher-accuracy regression with XGBoost	FUTURE	8	Phase 2
PBI-F03	Advanced Feature Engineering	Git diff size, dependency count, loop depth as ML features	FUTURE	5	Phase 2
PBI-F04	Real-Time Model Retraining	Auto-retrain on schedule using live production data	FUTURE	13	Phase 2
PBI-F05	AWS Pricing API Integration	Replace mocked prices with live AWS Cost Explorer data	FUTURE	8	Phase 3
PBI-F06	GCP Cost Estimation Support	Extend pricing simulation to Google Cloud Platform	FUTURE	8	Phase 3
PBI-F07	Azure Cost Support	Add Microsoft Azure pricing tier support	FUTURE	8	Phase 3
PBI-F08	Multi-Cloud Comparison Dashboard	Side-by-side cost comparison: AWS, GCP, Azure	FUTURE	13	Phase 3
PBI-F09	Custom Kubernetes Operator	K8s operator for proactive autoscaling from predictions	FUTURE	13	Phase 4
PBI-F10	Slack & Email Alerting	Real-time cost spike alerts via Slack and email	FUTURE	5	Phase 4
PBI-F11	Kafka Distributed Processing	Event streaming pipeline for high-throughput PR processing	FUTURE	13	Phase 4
PBI-F12	Cloud Deployment (AWS/GCP)	Deploy full system to cloud for production	FUTURE	8	Phase 4
PBI-F13	GitHub Marketplace Listing	Package ShadowStack as an installable GitHub App	FUTURE	8	Phase 4
PBI-F14	Multi-Language Code Analysis	Extend AST analysis to JavaScript, Go, and Java	FUTURE	13	Phase 4

Table 5: Future Backlog Items (PBI-F01 to PBI-F14)

Complete MVP Backlog Summary

ID	Sprint	Title	Priority	SP	Owner	Epic
PBI-001	1	System Architecture Diagram	HIGH	2	Aniruddha	EP-1
PBI-002	1	FastAPI Setup with PostgreSQL	HIGH	5	Aniruddha	EP-2
PBI-003	1	GitHub Webhook Listener Endpoint	HIGH	5	Aniruddha	EP-2
PBI-004	1	Docker Configuration	HIGH	3	Aniruddha	EP-1
PBI-005	1	Product Backlog & Sprint Planning	HIGH	2	Aniruddha	EP-3
PBI-006	1	React Project Initialization	HIGH	3	Sanay	EP-4
PBI-007	1	Dashboard Wireframes	HIGH	2	Sanay	EP-4
PBI-008	1	Basic UI Components (Header, Sidebar, Nav)	HIGH	3	Sanay	EP-4
PBI-009	1	Chart Library Setup	MED	2	Sanay	EP-4
PBI-010	1	Software Requirements Specification (SRS)	HIGH	3	Arpita	EP-3
PBI-011	1	Use-Case Diagrams	HIGH	2	Arpita	EP-3
PBI-012	1	Dataset Schema Design	HIGH	2	Arpita	EP-3
PBI-013	1	Sprint Tracking Documents & Burn-down Chart	HIGH	2	Arpita	EP-3
PBI-014	1	Unit Tests: Backend Core Modules	MED	2	Aniruddha	EP-10
PBI-015	1	Sprint 1 Retrospective Notes	MED	1	Arpita	EP-3
PBI-016	2	Synthetic Dataset Generation ($\geq$500 samples)	HIGH	5	Arpita	EP-5
PBI-017	2	Historical Usage Data Collection	HIGH	3	Arpita	EP-5
PBI-018	2	Data Preprocessing Utilities	HIGH	3	Arpita	EP-5
PBI-019	2	Sprint 2 Test Case Documentation	MED	2	Arpita	EP-3
PBI-020	2	AST-Based Python Code Parser	HIGH	5	Aniruddha	EP-2
PBI-021	2	Cyclomatic Complexity Scoring Algorithm	HIGH	5	Aniruddha	EP-2
PBI-022	2	Feature Extraction Pipeline	HIGH	5	Aniruddha	EP-5
PBI-023	2	Linear Regression Model Training	HIGH	3	Aniruddha	EP-5
PBI-024	2	ARIMA Time-Series Model Training	HIGH	5	Aniruddha	EP-5
PBI-025	2	Random Forest Ensemble Model Training	HIGH	5	Aniruddha	EP-5
PBI-026	2	Model Evaluation Framework (MAE/RMSE/R^2)	HIGH	3	Aniruddha	EP-5
PBI-027	2	Model Serialisation & Versioning	MED	2	Aniruddha	EP-5
PBI-028	2	Cost Simulation Engine with Mocked Pricing	HIGH	5	Aniruddha	EP-6
PBI-029	2	Basic Cost Estimation Logic	HIGH	3	Aniruddha	EP-6
PBI-030	2	Full RESTful API Endpoints	HIGH	5	Aniruddha	EP-7
PBI-031	2	Unit Tests: ML & Analysis Modules	HIGH	3	Aniruddha	EP-10
PBI-032	2	Sprint 2 Retrospective & Standup Logs	MED	1	Arpita	EP-3
PBI-033	3	API Integration Layer (Axios)	HIGH	3	Sanay	EP-8
PBI-034	3	Cost Trend Line Chart	HIGH	5	Sanay	EP-8

ID	Sprint	Title	Priority	SP	Owner	Epic
PBI-035	3	Service-Wise Cost Breakdown Pie Chart	HIGH	5	Sanay	EP-8
PBI-036	3	Predicted vs. Actual Cost Comparison Chart	HIGH	5	Sanay	EP-8
PBI-037	3	ML Model Performance Metrics Panel	HIGH	3	Sanay	EP-8
PBI-038	3	Historical Trends with Time-Range Filters	MED	3	Sanay	EP-8
PBI-039	3	Interactive Data Filtering & Sorting	MED	3	Sanay	EP-8
PBI-040	3	Real-Time Dashboard Updates	MED	3	Sanay	EP-8
PBI-041	3	Responsive UI & Accessibility	MED	3	Sanay	EP-8
PBI-042	3	Team Cost Dashboard View	MED	2	Sanay	EP-8
PBI-043	3	Cost History Retrieval Endpoint	HIGH	3	Aniruddha	EP-2
PBI-044	3	Frontend Unit Tests (Jest/RTL)	HIGH	3	Sanay	EP-10
PBI-045	3	Frontend Validation & Test Case Docs	MED	2	Arpita	EP-3
PBI-046	3	Sprint 3 Retrospective & Standup Logs	MED	1	Arpita	EP-3
PBI-047	4	GitHub PR Comment Automation	HIGH	8	Aniruddha	EP-9
PBI-048	4	End-to-End Workflow (PR to Dashboard)	HIGH	8	Aniruddha	EP-11
PBI-049	4	Performance Optimisation & Profiling	HIGH	3	Aniruddha	EP-11
PBI-050	4	Final Docker Deployment Verification	HIGH	2	Aniruddha	EP-11
PBI-051	4	API Documentation	HIGH	2	Aniruddha	EP-3
PBI-052	4	Integration Tests: Critical Workflows	HIGH	5	Arpita	EP-10
PBI-053	4	System Testing & Validation	HIGH	5	Arpita	EP-10
PBI-054	4	Deployment Guide & User Manual	HIGH	2	Arpita	EP-3
PBI-055	4	Final Project Report Compilation	HIGH	3	Arpita	EP-3
PBI-056	4	Comprehensive Bug Fixing	HIGH	3	All	EP-10
PBI-057	4	UI/UX Refinements & Final Polish	MED	2	Sanay	EP-11
PBI-058	4	Demo Preparation & Presentation Slides	HIGH	3	All	EP-11
PBI-059	4	Code Cleanup & Refactoring	MED	2	All	EP-11
PBI-060	4	Sprint 4 Retrospective & Lessons Learned	MED	1	Arpita	EP-3
			Sprint 1	**37**		
			Sprint 2	**63**		
			Sprint 3	**47**		
			Sprint 4	**49**		
			Total MVP Story Points	**196**		

Table 6: Complete MVP Backlog Summary — PBI-001 to PBI-060

Success Metrics and Definition of Done

Measurable Success Criteria

Metric	Target	Key Backlog Items
ML Model Accuracy (MAE)	<15% on test dataset	PBI-023 to PBI-026
PR Analysis Response Time	<30 seconds	PBI-003, PBI-048
Dashboard Load Time	<2 seconds	PBI-033 to PBI-044
System Uptime	≥95%	PBI-053
Code Test Coverage	≥70%	PBI-014, PBI-031, PBI-044, PBI-052
API Response Time (P95)	<500 ms	PBI-030, PBI-049
Documentation Completeness	100% of all modules	PBI-010, PBI-051, PBI-054, PBI-055
MVP Feature Completion	100% of PBI-001–060	All

Table 7: Measurable Success Criteria

Definition of Done (DoD)

A backlog item is **Done** when *all* of the following are true:

1. All acceptance criteria verified and signed off by the item owner.
2. Code merged to main branch via a reviewed GitHub pull request.
3. Relevant unit and/or integration tests written and passing in CI.
4. No new linting or type-checking errors introduced.
5. Feature is demonstrable in the running Docker environment.
6. Impacted documentation updated (README, API docs, or user guide).
7. Item marked as *Done* on the Agile board.

Live Demo — Required Scenarios

The final demonstration must successfully show all six scenarios:

1. Creating a GitHub pull request with real Python code changes.
2. System automatically analysing the code and posting a cost-prediction PR comment.
3. Dashboard displaying predicted vs. current costs updated in real time.
4. Service-wise cost breakdown donut chart.
5. Historical cost trends chart with time-range filtering.
6. ML model performance metrics panel showing MAE, RMSE, and R^2.

Living document. Sprint deadlines: S1 — Feb 24 | S2 — Mar 8 | S3 — Apr 5 | S4 — Apr 15, 2026. Sprint 1 deliverables sourced from the official Sprint 1 document.